GENESIS OS / RELATIONSHIP SCORES

# PHASE 1 MVP BUILD SPECIFICATION

Workbook-aligned launch document

**Why this version exists**

This document defines only what must be built for Phase 1 MVP.

It intentionally narrows launch scope while preserving the correct scoring engine.

The system was changed because it needed to be correct for launch, not because the prior version lacked vision.

| Status | Build-directive / send-ready developer spec |
|---|---|
| Primary public score in Phase 1 | CompatibilityScore (350-850) |
| Phase 1 principle | Implement the real engine now; calibrate later |
| Not included in Phase 1 | Full AI, full Sanctuary, couple mode, advanced drift, ML tuning |
| Source-of-truth stance | Workbook score law first; this Phase 1 spec controls launch scope |

*Prepared for developer alignment and launch planning. Proper full-system documentation package to be uploaded separately.*

## 1. Executive summary

Phase 1 MVP is not a simplified scoring concept. It is the minimum launchable implementation of the correct Genesis OS engine and the minimum product loop around that engine.

**Executive decision**

- Phase 1 MUST launch with the workbook-aligned deterministic compatibility engine.
- Phase 1 MUST NOT use the deprecated weighted shortcut formula from earlier blueprints.
- Phase 1 MUST keep scoring on the backend and use imported workbook artifacts as the authority.
- Phase 1 MUST reduce product surface area, not reduce engine correctness.
- Anything not required for launch should be deferred rather than half-built.

This specification exists because older blueprint packages were useful but transitional. They still carried overlapping logic, including an older weighted component model and newer AnswerPairQuantum doctrine in the same document family. That is acceptable for internal evolution, but it is not acceptable for launch execution. Phase 1 therefore freezes a narrow, developer-safe build path: implement the correct engine, expose only the minimum required product surfaces, and defer everything that does not need to ship on day one.

## 2. Why the system changed

The system changed because the scoring needed to be correct for launch. The older blueprint generation blended previous logic with the newer workbook-driven architecture. That created the risk that a team could build a partially correct system, then 'fix it later' after design and data assumptions were already embedded into the product.

That approach is not acceptable for Genesis OS because the scoring engine is the product moat. If the engine is watered down, front-end polish does not save the product. The launch architecture therefore moved fully to the deterministic workbook-first model:

- Answers are treated as vectors, not just as points on a simple weighted checklist.
- Pair interactions are resolved through AnswerPairQuantum lookups, not guessed on the fly from a loose front-end formula.
- Risk gating and rule penalties are part of the score law, not optional tuning layers.
- Polarity v10 is a real chemistry subsystem inside the score, not a decorative add-on.
- Readiness, Compatibility, and Forecast are separate products or output families and must not be casually blended.

**Plain-language rationale**

We changed the build doctrine for one reason: the system needed to be correct for launch.

Phase 1 therefore narrows scope around the real engine instead of launching a diluted version and promising a future rebuild.

## 3. Source-of-truth order for Phase 1

Developers must not choose between multiple legacy interpretations. Phase 1 uses the following source order:

| Priority | Artifact | Phase 1 rule |
|---|---|---|
| 1 | Authoritative workbook package | Controls score law, matrices, pair rows, overlays, and polarity package. |
| 2 | Current build-governing unified blueprint | Controls implementation narrative, storage, runtime order, and governance. |
| 3 | This Phase 1 MVP spec | Controls launch scope: what must ship now, what must wait, and what is intentionally deferred. |
| 4 | Older blueprint fragments or PDF notes | May inform context only when they do not conflict with items 1-3. |

## 4. Non-negotiable launch principles

| Principle | What it means in practice |
|---|---|
| Deterministic first | Every visible score must be reproducible from stored answers, imported tables, versioned formulas, and auditable logs. |
| Backend-only score law | The client may render inputs and results, but the client may not own score math, rule penalties, gate logic, or pair aggregation. |
| No client-side recreation | AnswerPairQuantum, question weighting, overlays, and pair score assembly must live on the server. |
| No monetized distortion | Coins, badges, premium status, vanity identity, or social layers may not alter score, rank, confidence, or eligibility. |
| Event-sourced truth | Raw answers and immutable events remain authoritative; later inferences sit on top of them and never erase them. |
| Graceful degradation | If optional AI or telemetry layers are unavailable, deterministic scoring must still return cleanly. |
| Consent by scope | Chat analysis, Sanctuary AI, and future pair AI are separate permissions and must not be assumed. |
| Launch scope discipline | If a feature is not required to launch the correct engine, defer it rather than shipping a partial version. |

## 5. What Phase 1 must deliver

Phase 1 succeeds only if a user can complete the assessment, have their answers stored in a versioned ledger, have the system compute a correct compatibility score against eligible candidates using the workbook-aligned pair engine, receive a basic pair report, and enter either direct messaging or Knock according to policy.

- Onboarding and profile setup with required identity, consent, and compatibility-input fields.
- 60-question assessment flow with versioned answer storage.
- Imported workbook data products and validation routines.
- Backend scoring service using the canonical pair-quantum sequence.
- Pair compute endpoint and persisted pair output records.

- Basic compatibility result surfaces for self and match preview.
- Basic access gating with direct message vs Knock logic.
- Event ledger, consent model, and calibration capture foundations.
- Admin or QA visibility into imports, score traces, and parity checks.

## 6. What Phase 1 explicitly does NOT need

Phase 1 is intentionally not the full Genesis OS vision. The MVP should preserve future expansion without forcing the team to ship everything at once.

- Full public Readiness product
- Full public Relationship Forecast family
- Advanced runtime drift engine and slow-trait mutation jobs
- Runtime shadow activation surfaces beyond storage hooks
- Full Sanctuary module suite beyond optional placeholders
- Quantum Mirror / premium AI reflection
- Couple mode, relationship containers, or Symphonic Match
- Games, coins, badges, social identity, or growth economy
- ML reweighting, autonomous tuning, rank-lift, or prediction models
- Rich moderation dashboards beyond minimum logging and safety stubs

**Scope rule**

Do not simplify the engine to make Phase 1 smaller.

Make Phase 1 smaller by reducing product surface area and deferring non-essential modules.

## 7. Phase 1 build-now / build-later scope matrix

| Domain | Phase 1 decision | Required now | Notes |
|---|---|---|---|
| Onboarding and profile | Build now | Yes | Required to collect identity, consent, and overlay inputs. |
| Assessment intake (60 questions) | Build now | Yes | Must store versioned answer selections. |
| AnswerBank import | Build now | Yes | Backend-owned import; no client scoring. |
| AnswerPairQuantum import | Build now | Yes | Core engine artifact; not optional. |
| Archetype and Shadow matrices | Build now | Yes | Needed for score law and diagnostics. |
| Polarity v10 package | Build now | Yes | Chemistry is part of compatibility, not a later cosmetic feature. |
| Zodiac and Numerology overlays | Build now | Yes | Use safe fallback if keys are missing. |
| Deterministic scoring service | Build now | Yes | Core launch deliverable. |
| Basic self blueprint output | Build now | Yes | Store aggregates and controlled internal fields. |
| Candidate generation | Build now | Yes | Can be simple, but must |

| | | | |
|---|---|---|---|
| | | | support pair compute. |
| Basic match report | Build now | Yes | Compatibility score, tier, translated drivers. |
| Direct message vs Knock gating | Build now | Yes | Thresholds must be config-managed. |
| Event ledger and calibration capture | Build now | Yes | Needed to avoid rebuilding later. |
| Admin parity / QA view | Build now | Yes | Required to validate imported logic. |
| Public readiness score | Defer | No | Store the ingredients only. |
| Public forecast bundle | Defer | No | Store hooks only unless separately approved. |
| Quantum Mirror / AI reflection | Defer | No | Phase 2+ after packet and consent law are stable. |
| Full Sanctuary suite | Defer | No | Keep placeholders if desired, but do not block launch. |
| Couple mode | Defer | No | Not required for Phase 1. |
| ML tuning and model-assisted ranking | Defer | No | Collect labels now; train later. |

# 8. Functional requirements - detailed Phase 1 build

## 8.1 Onboarding and profile

Phase 1 onboarding must collect the minimum fields required to compute, govern, and later audit compatibility.

- user_id (internal UUID)
- rs_code or equivalent searchable identity handle
- account_state and relationship_state
- date_of_birth and gender where the current overlay model requires those inputs
- location or search-radius fields needed for candidate retrieval
- dating status / intent fields
- ai_consent, chat_analysis_consent, and relevant feature-scope flags
- visibility state, hidden state, or ghost-mode style suppression flags
- astrology and numerology keys or enough birth data to derive them later

Phase 1 does not need deep profile surfaces. It does need correct data capture, clean identifiers, and future-safe storage. These fields should be stored in canonical tables, versioned where appropriate, and available to the scoring and packet-building services.

## 8.2 Assessment flow and answer storage

The assessment in Phase 1 is a required launch component. The system must collect 60 answers: 50 core and 10 shadow. The front end asks questions; the backend owns all scoring, persistence, and versioning.

- Persist user_id, assessment_session_id, question_number, answer_id, answer_text snapshot, phase, scoring_version, and submitted_at.
- Store answer selections exactly as chosen, even if the answer bank changes later.
- Do not map, score, or aggregate answers in the client.

- Support resumed assessment sessions if product policy requires save-and-return behavior.
- If a retake policy exists, it must create a new versioned assessment record rather than overwriting the historical one.

## 8.3 Workbook data products and imports

Phase 1 cannot be built off static notes alone. The workbook artifacts must be imported as governed data products with validation metadata.

| Sheet | Role in Phase 1 | Status |
|---|---|---|
| Schema | Canonical weights, gate law, penalties, output rules | Required |
| AnswerBank | Answer vectors, norms, labels, tags, question weights | Required |
| AnswerPairQuantum | Precomputed pair metrics, gate, rule_penalty, pair_norm | Required |
| ArchetypeMatrix | 10x10 archetype fit values | Required |
| ShadowMatrix | 6x6 shadow fit or risk values | Required |
| ZodiacLookup | Overlay lookup table | Required |
| NumerologyLookup | Overlay lookup table | Required |
| MatchHarness | QA parity and runtime sequence fixture | Required |
| Polarity_Axes_v10 | Ten-axis chemistry config | Required |
| Polarity_RoleSeeds_v10 | Role-seed anchors for auditability | Required |
| Polarity_QuestionMap_v10 | Prompt-to-axis mapping | Required |
| Polarity_AnswerBank_v10 | Per-answer polarity vectors | Required |
| Polarity_PairEngine_v10 | Chemistry formula registry | Required |
| FeedbackData | Calibration schema | Required for storage, not for live model tuning |

Every import must record workbook_version, sheet_name, sheet_rows, data_rows, imported_at, source_checksum, import_job_id, and score_contract_version. Phase 1 must validate imported counts rather than hard-code disputed magic numbers.

## 8.4 Deterministic scoring service

This is the core of Phase 1. The scoring service computes the official compatibility output. It is the only service allowed to author authoritative score values.

### 8.4.1 Runtime order

1. Normalize inputs: collect two users' selected answer IDs plus required identity keys such as gender, sun sign, and life-path values if overlays are enabled.
2. Build pair_key values: for each comparison, sort answer IDs into ans_low|ans_high so the join is order-insensitive.
3. Lookup pair rows: read component fits, risk fields, gate, rule_penalty, and pair_norm from AnswerPairQuantum.
4. Apply question weighting: multiply each selected pair contribution by question_weight_v3 or the equivalent imported weighting field.
5. Aggregate: compute CoreNorm and required subcomponent averages using weighted averages only.

6. Apply overlays: add zodiac and numerology only through their locked 7 percent / 5 percent overlay law.
7. Clamp and convert: clamp FinalNorm to [0,1] and convert to the public 350-850 scale.
8. Return score payload: return CompatibilityScore, tier, and translated drivers while hiding raw internal safety fields.

### 8.4.2 Canonical formulas

```
BehavioralComponent = 0.16*archetype_fit + 0.10*comm_fit + 0.10*reg_fit + 0.06*love_fit +
0.10*readiness_fit
StabilityComponent  = 0.09*shadow_fit + 0.07*integrity_fit + 0.04*cluster_fit +
0.03*pace_fit
ChemistryComponent  = 0.08*polarity_fit + 0.05*pursuit_fit
base_0_88           = BehavioralComponent + StabilityComponent + ChemistryComponent
gate                = 1 - (0.55*risk_avg + 0.20*risk_max)
pair_norm_0_1       = max(0, min(1, (base_0_88 * gate) - rule_penalty))
CoreNorm            = weighted average of pair_norm_0_1 across selected question pairs
CosmicOverlay       = 0.07*ZodiacNorm + 0.05*NumerologyNorm
FinalNorm           = clamp(CoreNorm + CosmicOverlay, 0, 1)
CompatibilityScore  = 350 + 500*FinalNorm
```

**Scoring clarification**

Phase 1 must implement this exact law or call the exact service that implements it.

Phase 1 must not use the older shortcut formula or any simplified client-side recreation.

Testing later means parity, calibration, and tuning - not postponing the real engine.

### 8.4.3 Required scoring outputs

| Field | Type | Visibility | Phase 1 use |
|---|---|---|---|
| CompatibilityScore | int (350-850) | Match-visible | Primary ranking score |
| Tier | enum | Match-visible | Access gating and basic explanation |
| FinalNorm | 0-1 float | Internal | Diagnostics and QA |
| CoreNorm | 0-1 float | Internal | Diagnostics and QA |
| StabilityAvg | 0-1 float | Internal | Diagnostics and future reports |
| ChemistryAvg | 0-1 float | Internal | Diagnostics and future reports |
| ZodiacNorm | 0-1 float | Internal | Overlay diagnostics |
| NumerologyNorm | 0-1 float | Internal | Overlay diagnostics |
| Top positive drivers | translated list | User-safe | Basic explanation |
| Top friction drivers | translated list | User-safe | Basic explanation |
| scoring_version | string | Internal | Reproducibility |

### 8.4.4 What the scoring service must not do

- Do not score from raw answer text.
- Do not make pair joins order-sensitive.
- Do not average questions equally unless the imported weight says so.
- Do not let cosmic overlays dominate weak core fit.

- Do not expose raw shadow severity, hidden moderation data, or raw risk flags to other users.
- Do not recreate the engine in Bubble, web, or mobile code.

### 8.5 Basic self blueprint output

Phase 1 should produce and store a basic self blueprint, but it does not need to launch the full future self-product stack. The purpose is to support matching, future readiness, and controlled diagnostics.

| Required stored aggregate | Purpose in Phase 1 |
|---|---|
| communication average | self report + future scoring diagnostics |
| regulation average | self report + future readiness / forecast |
| love or relational quality average | self report + future diagnostics |
| readiness-aligned average | future self product / internal QA |
| pacing and attachment-risk averages | future pair interpretation |
| integrity and cluster burden rollups | future rule support and diagnostics |
| archetype baseline and shadow baseline | matching and future reports |
| polarity baseline vector | chemistry and future explanation |

Phase 1 may show a basic self blueprint summary, but any public Readiness product is deferred unless separately approved. The system should store the ingredients now so launch does not create future rework.

### 8.6 Candidate generation and pair compute

Phase 1 does not require a sophisticated recommendation layer. It does require a reliable way to retrieve candidate users who pass basic filters and then compute deterministic pair outputs.

- Retrieve candidates using location, visibility, relationship state, and product eligibility filters.
- Assemble a pair packet from two self records and required policy masks.
- Run pair compute on demand or through controlled precompute jobs.
- Persist pair_id, user_a_id, user_b_id, CompatibilityScore, Tier, FinalNorm, CoreNorm, StabilityAvg, ChemistryAvg, overlay norms, scoring_version, and computed_at.
- Store pair diagnostics separately from public presentation so report filtering can hide internal-only fields.

### 8.7 Access gating, Knock, and basic messaging

Phase 1 must support the minimum access flow between scored users. The product may open direct messaging or require a Knock flow depending on a config-managed threshold policy.

- Direct message eligibility and Knock-required status must be returned by the access engine, not guessed in the client.
- Thresholds must be config-managed and versioned; do not hard-code values inside the app.
- A Knock record should include sender, target, timestamp, state, and outcome.
- Basic messaging should store conversations, messages, timestamps, and consent-relevant metadata.
- If chat analysis is not consented, Phase 1 must still log operational events but must not read private content for AI or behavioral extraction.

## 8.8 Event ledger and telemetry foundation

Phase 1 does not need advanced machine learning, but it does need the event spine that will make later calibration and forecasting possible.

- answer_submitted
- assessment_completed
- assessment_scored
- match_computed
- knock_sent
- knock_accepted or knock_declined
- message_sent
- conversation_opened
- status_changed
- consent_changed
- pair_report_viewed
- optional feedback or outcome checkpoint events

The event ledger must be append-only. Derived outputs such as self blueprints, pair computations, and later feature-store rows should be reconstructable from source events and imported score versions.

## 8.9 Calibration capture (store now, tune later)

The dev team does not need to prove calibration quality before Phase 1 launch. It does need to capture the fields that will make calibration possible later.

| Telemetry field | Why it must exist in Phase 1 |
|---|---|
| match_score / tier_label | future cohort analysis and quality review |
| message_count_7d | engagement proxy |
| reply_latency_p50 | responsiveness proxy |
| date_first_meet | conversion signal |
| relationship_status_30d and 90d | outcome labels |
| breakup_reason or outcome reason | later penalty tuning |
| conflict_events_count and repair_events_count | stability truth signals |
| user_reported_satisfaction / safety / chemistry | future supervised labels |

This is where 'testing later' belongs: later testing is about distribution, fit to outcomes, and tuning. It is not about deciding whether the engine gets implemented.

## 8.10 Admin and QA visibility

Phase 1 needs enough internal visibility to know whether the team built the right system. Without this, the app can appear to work while silently drifting away from score law.

- Import audit screen showing versions, row counts, checksums, and import timestamps.
- Score trace view showing answer IDs, pair keys, selected pair rows, gate, penalties, weighted aggregation, and final outputs.
- Parity fixture runner using frozen MatchHarness scenarios.
- Visibility test tools to confirm no blocked fields are leaking to self, match, or future AI routes.
- Recompute route or admin job that can replay scores against a selected version package.

## 9. Phase 1 data model and storage requirements

The following tables or objects are the minimum practical storage footprint for a correct Phase 1 build. Names may vary by implementation, but the object boundaries should remain.

| Domain | Core tables or objects | Why they exist in Phase 1 |
|---|---|---|
| Identity and consent | users, consent_state, visibility_settings, entitlement records | identity, privacy, policy gating |
| Assessment and imports | assessment_sessions, questions, answer_bank, answer_pair_quantum, matrices, overlays, polarity tables | score authority and reproducibility |
| User outputs | user_answers, user_profile_aggregates, self_blueprint_outputs | self diagnostics and matching inputs |
| Pair outputs | pair_computations, pair_diagnostics, access_decisions | match outcomes, explainability, gating |
| Messaging and access | knock_records, conversations, messages | basic communication flow |
| Event ledger | event_ledger | append-only truth and rebuild path |
| Feedback and telemetry | outcome_checkpoints or equivalent calibration tables | future tuning and QA |
| Feature placeholders | feature_store or future-ready packet tables | avoid rebuild later, even if sparsely used at launch |

## 10. Phase 1 API surface

Endpoints may be implemented under different route names, but the platform needs these functional contracts:

| Endpoint | Owned by | Returns | Phase 1 note |
|---|---|---|---|
| POST /assessment/submit | Gateway -> Scoring | self blueprint payload | primary completion route |
| GET /profile/score | Gateway | public-safe self payload | read-only path |
| POST /match/compute | Gateway -> Matching | match preview or full report | checks reveal / unlock / policy |
| POST /knock/send | Gateway -> Access/Messaging | knock outcome | for restricted access routes |
| POST /messages/send | Gateway -> Messaging | message ack | stores metadata and content under consent |
| GET /packets/self | Packet builder | self packet JSON | internal/admin use |
| GET /packets/pair | Packet builder | pair packet JSON | internal/admin use |
| POST /admin/recompute-score | Admin tools | recompute result | internal only |
| POST /training/export | Packet builder + feature store | dataset manifest | restricted internal route; may be stubbed |

## 11. Performance, reliability, and governance requirements

- Synchronous scoring routes must feel consistent and trustworthy; optional heavy jobs belong in queues or workers.
- The system must return deterministic compatibility even if optional AI, telemetry enrichment, or future genome extraction is unavailable.

- Malformed packets must be rejected at the gateway; downstream services may not guess missing fields.
- Cache only public-safe or policy-safe artifacts. Do not cache raw sensitive text in unsafe layers.
- Every score and report must carry a scoring_version and enough metadata to be reproduced later.
- Any configuration that affects thresholds, gating, or visibility must be versioned and auditable.

## 12. Detailed acceptance criteria

Phase 1 is accepted only when the following conditions are true:

- At least five frozen MatchHarness scenarios produce the exact expected score outputs across environments.
- AnswerPairQuantum lookup is order-insensitive for every tested pair_key.
- The score service produces identical results when the same imports, inputs, and versions are used.
- No compatibility calculation occurs in the client.
- Public-safe surfaces hide raw shadow severity, raw safety flags, raw moderation artifacts, and raw internal cluster enforcement fields.
- If AI is unavailable, deterministic scores still return cleanly.
- If cosmic keys are missing, the service uses the approved safe fallback without blocking scoring.
- Every displayed score is traceable to input window, scoring version, policy version, and source checksum.
- Premium or cosmetic layers cannot influence score, rank, access thresholds, or confidence.
- The team can inspect imports, parity cases, and score traces without reverse-engineering production data.

## 13. Clarification on scoring complexity and 'testing later'

The scoring side is absolutely more complex than a normal app feature. That does not mean the score law should be postponed. It means the team must separate implementation from calibration.

| Build now | Test or tune later |
|---|---|
| Import workbook artifacts correctly | Distribution review and histogram tuning |
| Implement pair lookup, gate, penalty, and aggregation exactly | Outcome-based calibration once real labels exist |
| Prove parity against MatchHarness fixtures | Sensitivity testing and cohort analysis |
| Store telemetry and event truth | ML or reweighting experiments |
| Version every score and import | Forecast and rank-lift models |

If a team says it can only build a simplified score now and revisit the real engine later, that is not a Phase 1 request - that is a proposal to launch the wrong system and rebuild it later. This specification does not approve that path.

## 14. Recommended build sequence for the dev team

**Workstream 1 - imports and score authority:** Import workbook config tables, load AnswerBank and AnswerPairQuantum, validate row counts and checksums, load matrices and polarity package.

**Workstream 2 - assessment and self packet:** Build onboarding, assessment intake, answer ledger, and user aggregate generation.

**Workstream 3 - scoring and pair compute:** Implement backend score service, pair packet assembly, pair output persistence, and score trace logging.

**Workstream 4 - match and access flow:** Build candidate retrieval, match preview, direct-message vs Knock gating, and basic messaging.

**Workstream 5 - event ledger and telemetry:** Stand up append-only events and minimum calibration capture.

**Workstream 6 - QA and admin parity:** Freeze MatchHarness scenarios, add import audit views, run parity tests, and verify exposure controls.

## 15. Out-of-scope build warnings

- Do not replace AnswerPairQuantum with a loose front-end formula.
- Do not expose raw shadow, moderation, or safety fields to matches.
- Do not let coins, badges, premium tiers, or vanity codes influence score or rank.
- Do not launch public readiness or forecast surfaces unless their versioned contracts are explicitly approved.
- Do not let one reflection, one conflict, or one bad week rewrite slow identity.
- Do not turn 'we will test later' into a reason to postpone the correct engine.

## 16. Developer handoff statement

This Phase 1 MVP specification is intended to remove ambiguity. The team should build the real engine now and keep the launch surface small. The older blueprint generations were useful for product evolution, but this launch path exists because the scoring needed to be correct for launch.

Proper full-system documentation package will be uploaded separately for the broader Genesis OS vision. Until then, this document should be treated as the Phase 1 launch directive: implement the workbook-aligned engine, preserve future expansion paths, and defer anything that is not required to ship the correct core system.

## Appendix A. Normalized import validation counts

Validate imported counts at build time; do not hard-code disputed magic numbers.

| Sheet | Expected imported shape | Phase 1 use |
|---|---|---|
| AnswerBank | 311 sheet rows / 310 scored data rows | versioned answer registry |
| AnswerPairQuantum | 48,206 sheet rows / 48,205 pair rows | canonical pair lookup |
| ArchetypeMatrix | 11 sheet rows / 10x10 usable matrix | exact fit lookup |
| ShadowMatrix | 7 sheet rows / 6x6 usable matrix | exact fit or risk lookup |
| Polarity_Axes_v10 | 11 sheet rows / 10 axes | official chemistry dimensions |
| Polarity_RoleSeeds_v10 | 6 sheet rows / 5 role seeds | scoring and content anchors |
| Polarity_QuestionMap_v10 | 33 sheet rows / 32 mapped prompts | question metadata |
| Polarity_AnswerBank_v10 | 161 sheet rows / 160 rows | polarity answer config |
| Polarity_PairEngine_v10 | 22 sheet rows / 21 data rows | chemistry formulas |

## Appendix B. Minimum public and internal output split

| Surface | May show in Phase 1 | Must stay hidden or internal |
|---|---|---|
| Self profile | basic blueprint summary, optional safe diagnostics | raw risk flags, raw moderation data, internal enforcement codes |
| Match preview | CompatibilityScore, tier, translated drivers | raw shadow severity, raw penalties, private partner data |
| Admin / QA | full score trace, versions, parity data | only legally or policy-restricted safety content |

## Appendix C. Deferred modules register

- Full public Readiness product
- Full public Relationship Forecast family
- Advanced runtime drift engine and slow-trait mutation jobs
- Runtime shadow activation surfaces beyond storage hooks
- Full Sanctuary module suite beyond optional placeholders
- Quantum Mirror / premium AI reflection
- Couple mode, relationship containers, or Symphonic Match
- Games, coins, badges, social identity, or growth economy
- ML reweighting, autonomous tuning, rank-lift, or prediction models
- Rich moderation dashboards beyond minimum logging and safety stubs